Semantic tagging

The OpenBIM and Haystack plan · built Nov 13, 2026

The OpenBIM and Haystack plan

1. Executive Summary & Objective

The goal of this initiative is to transition the Temco Controls software ecosystem (**T3000**) from a legacy configuration utility into a modern, open-standard **Building Operating System (BOS)**. By integrating **OpenBIM** standards (IFC via **Bonsai BIM** and **thatOpen**) and **Semantic Tagging** (Project Haystack) directly into our core application architecture, we will provide an automated, zero-configuration solution designed specifically for high-density, mission-critical infrastructure clients like **PLC Group** (Data Centers/Telecom).

High-Level Architectural Flow

1. **Physical/Spatial Input:** Bonsai BIM (Desktop Authoring via Blender Engine) builds/modifies a completely open, native ISO graph database (`.ifc`).
2. **Web Rendering & Asset Tracking:** `thatOpen` framework streams and renders high-density geometry and structural data directly inside browser-based environments via WebAssembly.
3. **Semantic Layer Integration:** Project Haystack tags are stored *onboard* the hardware non-volatile storage (NVS) and synchronized through the local database.
4. **Autonomous Execution:** Central AI engines (e.g., PLC's *AxIn*) ingest the resulting structured data model via a standardized, firewall-friendly REST/JSON API.

---

2. Component Mapping & Repositories

This plan targets the convergence of two primary Temco Controls open-source development assets:

- **`temcocontrols/T3000Webview`**: The modern web-centric front-end built on Quasar/Vue, supported by a high-performance local Rust service API.
- **`temcocontrols/T3000_Building_Automation_System`**: The mature, core desktop application engine running on C++/MFC.

---

3. Phased Implementation Roadmap

Phase 1: Database & Firmware Extension (The Foundation)

To achieve true onboard edge autonomy, metadata must live as close to the physical I/O as possible.

- **Local Storage Schema Update:** Modify the embedded SQLite schema managed by the `T3000Webview` Rust API to add a dedicated `haystack_tags` string field for all points, variables, and registers.
- **Firmware Modbus/BACnet Mapping:** Allocate a section of the **64KB NVS partition** on the **ESP32-S3** hardware controllers (adjacent to the Matter credential slots) to hold compressed tag strings.
- **C++ Core Integration:** Update the `T3000_Building_Automation_System` C++ engine to serialize and push these tag strings directly to the physical devices during standard programming updates.

Phase 2: WebUI Evolution & Smart Automation (T3000Webview)

Leverage web spreadsheet components to eliminate the engineering bottleneck of manual data configuration.

- **The Semantic Grid:** Integrate a "Haystack Tags" column into the Quasar grid layout (`localhost:3003/#/modbus-register`).
- **Heuristic Matching Engine ("Auto-Tagging"):** Build a client-side JavaScript utility within the web app that parses user descriptive names. For example, a point named `DAT_01` automatically resolves and applies the following semantic tags: `point` , `air` , `discharge` , `temp` , `sensor` .
- **Spatial Canvas Drawer Integration:** Enhance the visual configuration environment. When a technician drags and drops a temperature sensor inside the visual bounding box of an Air Handling Unit (AHU), the interface must automatically append the structural relational tag (e.g., `equipRef: "AHU-1"`) directly into that point's metadata array.

Phase 3: The Inter-Process Communication Loop

Maintain strict state alignment between the C++ native window wrapper and the embedded web container.

- **Web-to-Native Event Push:** When changes are made inside the `T3000Webview` grid, dispatch a structured JSON message across the existing messaging bridge:

```JSON
{
  "action": "update_tags",
  "panelId": 144,
  "entryIndex": 1,
  "tableType": "INPUT",
  "tags": "sensor temp air discharge unit:"°C""
}
```

- **Native Interception:** Update the C++ window message handler in `T3000_Building_Automation_System` to capture this event, commit the update to the local system `.db` / `.prog` file, and stage the controller configuration block for transmission.

Phase 4: OpenBIM Engine Integration (Bonsai BIM + thatOpen)

Transition from static 2D isometric graphics to a native, interactive 3D informational model.

- **Authoring & Asset Modeling (Bonsai BIM):** Utilize the desktop-level Python/Blender environment to map critical infrastructure assets (Server Racks, CRAC Units, DC Power Plants, UPS Battery Strings) using native IFC standards.
- **Web Stream Pipeline (thatOpen):** Integrate the web-native `@thatopen/components` framework into `T3000Webview`. This allows the application to stream, parse, and render compressed web-IFC geometry fragments directly inside a browser tab at native desktop speeds.
- **Semantic Binding:** Link the `GlobalId` (GUID) of the physical IFC object explicitly to the corresponding Haystack `equipRef` or `point` record in the database.

Phase 5: Enterprise REST API Layer (The Open Handshake)

Enable a direct data pipeline to cloud analytics platforms, completely bypassing heavy UI management steps.

- **Expose External Network Interfaces:** Configure the lightweight Rust microservice embedded within `T3000Webview` to safely bind to external network adapters, expanding past localhost constraints.
- **Standardized Haystack Operations:** Build endpoints compliant with the core Project Haystack REST protocol:
  - `POST /api/haystack/read` -> Queries the SQLite points table using standard semantic tags.
  - `POST /api/haystack/hisRead` -> Extracts raw data history and trend logs directly from the local file system.
- **Interactive Documentation (Swagger / OpenAPI):** Mount an interactive `/swagger` interface on the Rust microservice. This allows enterprise engineering teams to immediately discover endpoints, run validation scripts, and integrate data straight into modern AI engines.

4. Key Target Verticals & Specialized Templates

Data Centers (High-Density Power & Thermal Asset Sets)

- **Server Racks:** `rack`, `equip`, `it`, `power`, `pdu`, `temp`
- **CRAC Precision Cooling Units:** `hvac`, `cooling`, `ahu`, `inlet`, `exhaust`, `fan`, `temp`
- **UPS Systems:** `elec`, `pdu`, `ups`, `voltage`, `current`

Telecommunications (Dispersed Remote Edge Infrastructure)

- **DC Power Plants:** `elec`, `dc`, `plant`, `voltage`, `current`, `charge`
- **Battery Banks:** `battery`, `dc`, `equip`, `temp`, `voltage`, `health`

---

5. Strategic Competitive Advantage

By combining open-source design tools with firmware-level semantic tagging, Temco Controls bypasses the closed licensing barriers and specialized technician dependencies typical of legacy vendors (such as Honeywell Forge or Johnson Controls OpenBlue). The data from our hardware remains entirely open, self-describing, and ready for automated integration directly into the customer's choice of advanced analytics platforms from day one.